

A visible control plane for governed software delivery

Ten canonical screens. Four journeys. One persistent project.
State → reason → evidence → action → traceability.

Control-plane map first. Contextual detail one click away.

Grounded in Architecture & Implementation Plan v1.2 and Technical Implementation Specification v1.0.
Proposed designs and illustrative SupportDesk fixtures; no live agent or delivery commands.

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / PROJECT OVERVIEW

Project Overview

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ✓ IMPACT ANALYSIS ✓ PLANNING ● DEVELOPMENT ○ INTEGRATION ○ ASSURANCE ○ RELEASE ○ COMPLETE

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

RELEASED BASELINE

R1

9e31ab7

ACTIVE OBJECTIVE

DC-003

Feature Change · Development

NEXT OPERATOR ACTION

Inspect

TASK-106 waits on dependency

Active delivery cycle

Add LOW, MEDIUM and HIGH ticket priority

DC-003 · target R2 · candidate code remains provisional

Open control-plane map

Project continuity

CYCLE	INTENT	OUTCOME
DC-001 · Greenfield	Build SupportDesk	R1
DC-002 · Brownfield	Understand existing repository	Trusted baseline + readiness

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / DELIVERY CYCLE

Add ticket priority

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ✓ IMPACT ANALYSIS ✓ PLANNING ● DEVELOPMENT ○ INTEGRATION ○ ASSURANCE ○ RELEASE ○ COMPLETE

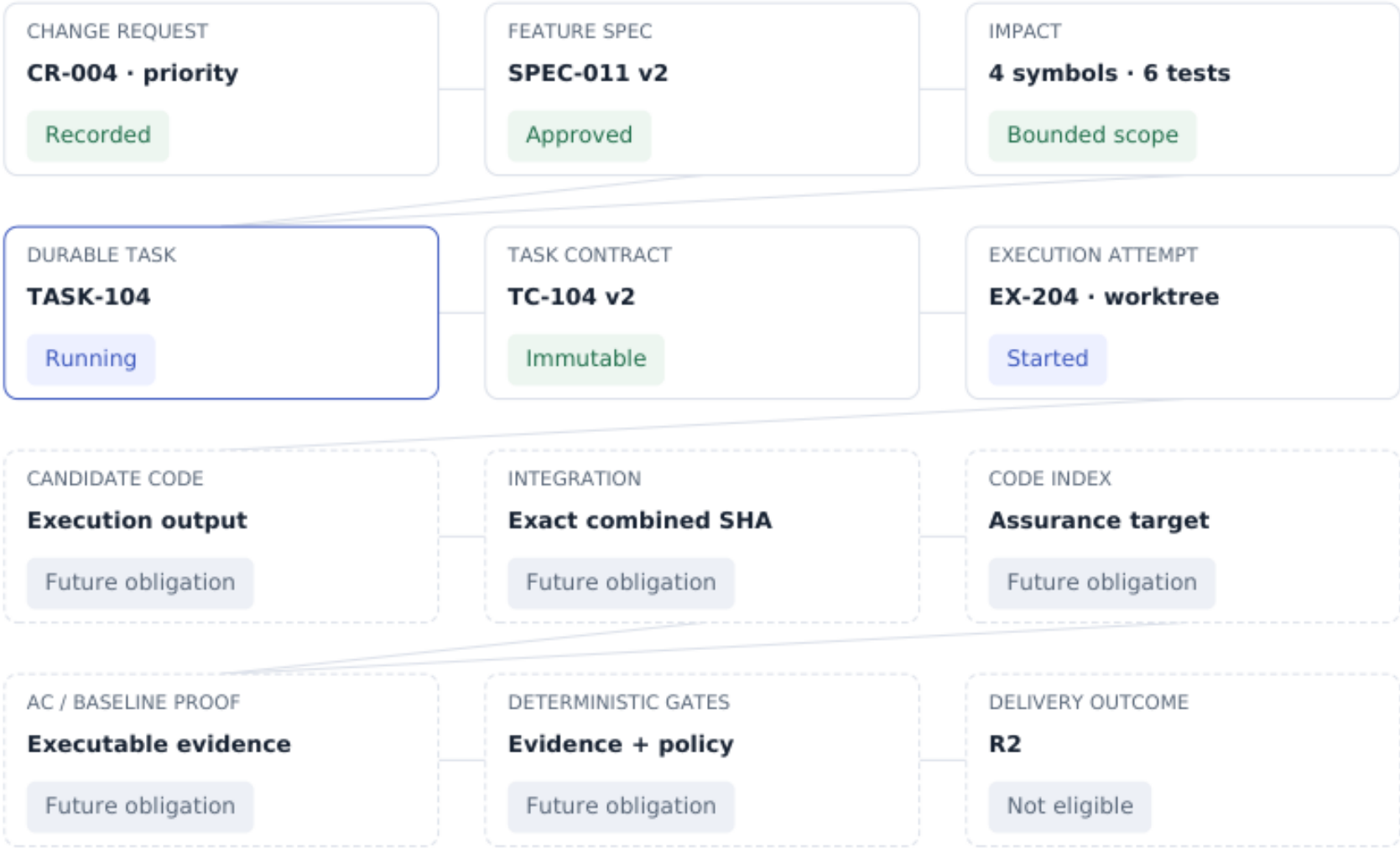
TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Control-plane map

Lifecycle is above; object relationships are the workspace



Select object → inspect reason / provenance → open contextual detail

Object inspector

Version-bound context and reason

TASK-104

Implement ticket priority in bounded scope

Authority

Olympus control-plane record

Version / origin

TC-104 v2 · SNAP-204

Commit scope

9e31ab7

Workspace

olympus/EX-204

Why this state?

A writable attempt is admitted only after dependencies, versions, scope and policy pass.

Open contextual detail

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / PRODUCT / SPECS

Product / Specs

SPEC DELTA

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE

● SPEC DELTA

○ IMPACT ANALYSIS

○ PLANNING

○ DEVELOPMENT

○ INTEGRATION

○ ASSURANCE

○ RELEASE

○ COMPLETE

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Behavioural specification

Stable feature identity · immutable approved versions

SPEC-011 v2

Ticket priority

FeatureSpec v1

subject
description
status = OPEN

FeatureSpec v2

subject
description
status = OPEN
+ priority: LOW / MEDIUM / HIGH
+ default: MEDIUM

CRITERION

REQUIRED PROOF

AC-011-03

E2E evidence against integrated SHA

Default status remains OPEN

Regression baseline

Review specification

Technical realization

ImplementationSpec v2

TicketModel · TicketService · API
Persistence schema + request contract
Unit / integration / E2E proof

Project architecture

ARCH-001
API → Service → Repository → PostgreSQL

A feature delta cannot silently redefine architecture.

Inspect impact scope

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / TASK DAG

Task DAG

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ✓ IMPACT ANALYSIS ✓ PLANNING ● DEVELOPMENT ○ INTEGRATION ○ ASSURANCE ○ RELEASE ○ COMPLETE

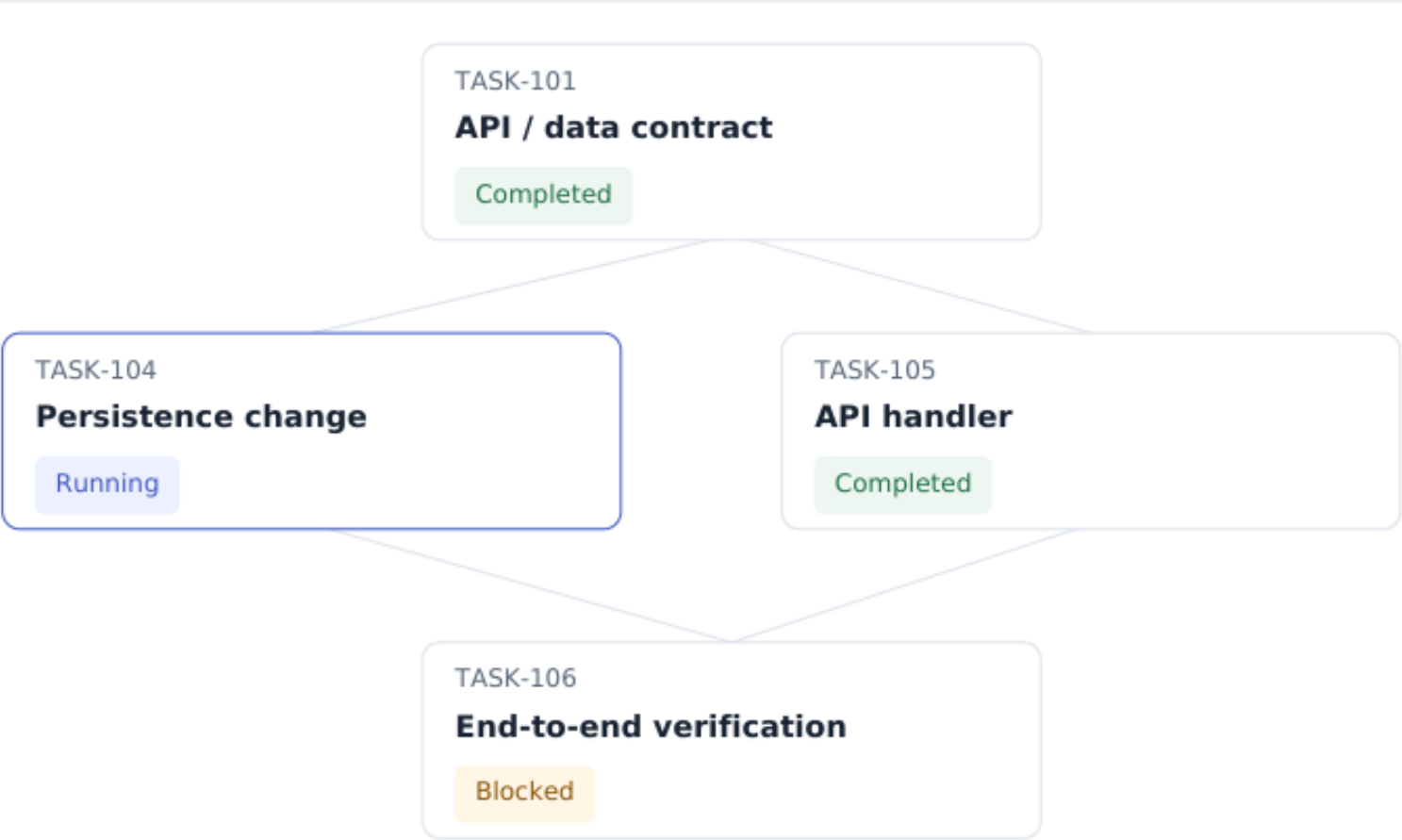
TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Task dependency graph

Durable planned work is distinct from each attempt



TASK-106 depends on TASK-104 + TASK-105

Explain blocked task

started · both attempts retained

TaskContract · TC-104 v2

Objective
Implement ticket priority

Allowed paths
app/models/*
app/services/*
tests/tickets/*

Required outputs
Candidate commit · changed files · tests

Prohibited
Authentication changes
Release branch writes

Open execution

WORKSPACE

01

Project Overview

02

Delivery Cycle

03

Product / Specs

04

Task DAG

05

Execution Inspector

06

Code Intelligence

07

Traceability

08

Impact Explorer

09

Assurance

10

Release

Integrations

Audit history

SS

Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / EXECUTION INSPECTOR

Execution Inspector

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE

✓ SPEC DELTA

✓ IMPACT ANALYSIS

✓ PLANNING

● DEVELOPMENT

○ INTEGRATION

○ ASSURANCE

○ RELEASE

○ COMPLETE

DEVELOPMENT

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

EX-204 · bounded execution

TC-104 v2 · SNAP-204 · base 9e31ab7

	STARTED	Forge · LangGraphRuntime	
10:42	Snapshot and lease created	platform	
10:43	Worktree: olympus/EX-204	git.worktree	
10:44	Allowed file inspected	repo.read	
10:47	Unit tests failed; evidence retained	test.run	
10:49	Implementation corrected in allowed scope	repo.write	
10:52	Unit tests passed	test.run	
10:53	Working changes; commit pending	candidate	

Inspect contract

Checkpoint / cancel

Execution boundary

Lease
worker-02 · heartbeat active

Snapshot hash
sha256:8c21...17ae

Model policy
implementation → configured alias

Permitted tools
repo.read / write · test.run
git.diff / commit

Each action crosses ToolGateway before
execution.

Open attempt lineage

WORKSPACE

- 01 Project Overview
- 02 Delivery Cycle
- 03 Product / Specs
- 04 Task DAG
- 05 Execution Inspector
- 06 Code Intelligence
- 07 Traceability
- 08 Impact Explorer
- 09 Assurance
- 10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / CODE INTELLIGENCE

Code Intelligence

INTEGRATION

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ✓ IMPACT ANALYSIS ✓ PLANNING ✓ DEVELOPMENT ● INTEGRATION ○ ASSURANCE ○ RELEASE ○ COMPLETE

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Code intelligence

Pinned canonical assurance index · c83a12d

CANONICAL ASSURANCE TARGET

app / services / ticket_service.py

TicketService.update_priority

def update_priority(ticket, priority):
 validate_priority(priority)
 ticket.priority = priority
 return repository.save(ticket)

RELATION	TARGET
IMPLEMENTS	SPEC-011 v2
CALLS	TicketRepository.save
VERIFIED_BY	test_update_priority
Illustrative symbol excerpt · not extracted code	
ORIGIN	GENERATED_LINEAGE · TASK-104 / EX-204

Repository truth

Released baseline
R1 · 9e31ab7

Current assurance target
IC-003 · c83a12d

Candidate worktree
EX-204 · provisional 4a871dc

These scopes are distinct. Only proof on the exact integration SHA can satisfy final gates.

Trace symbol to intent

WORKSPACE

- 01 Project Overview
- 02 Delivery Cycle
- 03 Product / Specs
- 04 Task DAG
- 05 Execution Inspector
- 06 Code Intelligence
- 07 Traceability
- 08 Impact Explorer
- 09 Assurance
- 10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / TRACEABILITY

Traceability

DEVELOPMENT

Add LOW, MEDIUM and HIGH ticket priority

- ✓ INTAKE
- ✓ SPEC DELTA
- ✓ IMPACT ANALYSIS
- ✓ PLANNING
- DEVELOPMENT
- INTEGRATION
- ASSURANCE
- RELEASE
- COMPLETE

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Narrative lineage

Every record and relationship is traversable in both directions

ORIGIN

CR-004 · requested ticket priority

SPECIFICATION

SPEC-011 v2 → AC-011-03

PLANNED WORK

ImplementationSpec → TASK-104 → TC-104 v2

EXECUTION

EX-204 → SNAP-204 @ 9e31ab7

CODE

Candidate 4a871dc → IC @ c83a12d

PROOF

Test / evidence → gate → approval

OUTCOME

R2 · pending manifest / eligibility

Why does this code exist?

This code realizes SPEC-011 v2 through an approved implementation boundary and attributed execution.

Relationship origin
GENERATED_LINEAGE

Brownfield alternative
DISCOVERED + confidence + source evidence;
HUMAN_CONFIRMED after review.

Failed attempts and superseded input versions remain reachable.

Inspect provenance

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / IMPACT EXPLORER

Impact Explorer

IMPACT ANALYSIS

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ● IMPACT ANALYSIS ○ PLANNING ○ DEVELOPMENT ○ INTEGRATION ○ ASSURANCE ○ RELEASE ○ COMPLETE

TASK-106 waits for TASK-104 completion

State → reason → evidence → allowed action → lineage

Why this state?

Impact explorer

Spec delta → structural paths → bounded change and verification

STRUCTURAL IMPACT

Ticket priority affects 4 principal symbols

PATH / OBLIGATION	SELECTION RATIONALE
SPEC-011 → TicketService	Persisted IMPLEMENTS link
TicketService → Repository	CALLS relationship
Repository → tickets schema	ACCESSES relationship
test_update_priority	VERIFIED_BY relation
BL-001 / BL-002 / BL-003	Preserve impacted existing behaviour

Review bounded plan

Proposed scope

app/models/ticket.py
app/services/ticket_service.py
app/api/tickets.py
tests/tickets/

Risk: MEDIUM
Architecture delta: none

Structural paths explain direct impact. Bounded semantic expansion is labelled as inferred impact and carries rationale.

Open task plan

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / ASSURANCE

Assurance

ASSURANCE

Add LOW, MEDIUM and HIGH ticket priority

✓ INTAKE ✓ SPEC DELTA ✓ IMPACT ANALYSIS ✓ PLANNING ✓ DEVELOPMENT ● ASSURANCE ○ RELEASE ○ COMPLETE

Release blocked: mandatory exact-SHA evidence and required approval are pending

State → reason → evidence → allowed action → lineage

Why this state?

Exact-SHA evidence matrix

IC-003 · c83a12d · all final proof targets this commit

SENTINEL INCOMPLETE

OBLIGATION	EVIDENCE	RESULT
AC-011-03	No required E2E result	MISSING
New unit behaviour	EV-501 @ c83a12d	PASS
Impacted baselines	EV-601 / 602 / 603	PASS
Engineering review	Warden evidence @ c83a12d	PASS

Old-SHA evidence remains history and cannot satisfy the current target. Model assertion cannot substitute for mandatory executable proof.

Inspect blocking proof

Request verification

Deterministic gate finalization

- ✓ Current index matches candidate
- ✓ Independent engineering review passed
- Mandatory acceptance evidence complete
- ✓ Blocking findings = 0
- Required approvals present

Finding → remediation Task → new Execution → integration → targeted verification

Open release eligibility

WORKSPACE

- 01 Project Overview
- 02 Delivery Cycle
- 03 Product / Specs
- 04 Task DAG
- 05 Execution Inspector
- 06 Code Intelligence
- 07 Traceability
- 08 Impact Explorer
- 09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-003 · Feature Change

New delivery cycle

SUPPORTDESK / RELEASE

Release

ASSURANCE

Add LOW, MEDIUM and HIGH ticket priority

- ✓ INTAKE
- ✓ SPEC DELTA
- ✓ IMPACT ANALYSIS
- ✓ PLANNING
- ✓ DEVELOPMENT
- ✓ INTEGRATION
- ASSURANCE
- RELEASE
- COMPLETE

Release blocked: mandatory exact-SHA evidence and required approval are pending

State → reason → evidence → allowed action → lineage

Why this state?

R2 · versioned outcome manifest

Release is independent of optional deployment

NOT ELIGIBLE		Assurance target: c83a12d
ELIGIBILITY CONDITION		RESULT
Manifest + current candidate		PASS
Required executions not stale		PASS
Warden + blocking findings		PASS
Mandatory AC / baseline evidence		AC-011-03 E2E MISSING
Human release approval		PENDING

Review release approval

Execute release · blocked

Release truth

Canonical assurance target

c83a12d

Current released baseline

R1 · 9e31ab7

Deployment

Not requested

Approval cannot substitute for proof. Eligibility includes all required approvals. Only a server-accepted execute command records a release.

Open complete lineage

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-001 · Greenfield Build

New delivery cycle

SUPPORTDESK / DELIVERY CYCLE

Delivery Cycle

Build SupportDesk from product intent

✓ DISCOVERY

● PRODUCT MODEL

○ ARCHITECTURE

○ PLANNING

○ DEVELOPMENT

○ INTEGRATION

○ ASSURANCE

○ RELEASE

○ COMPLETE

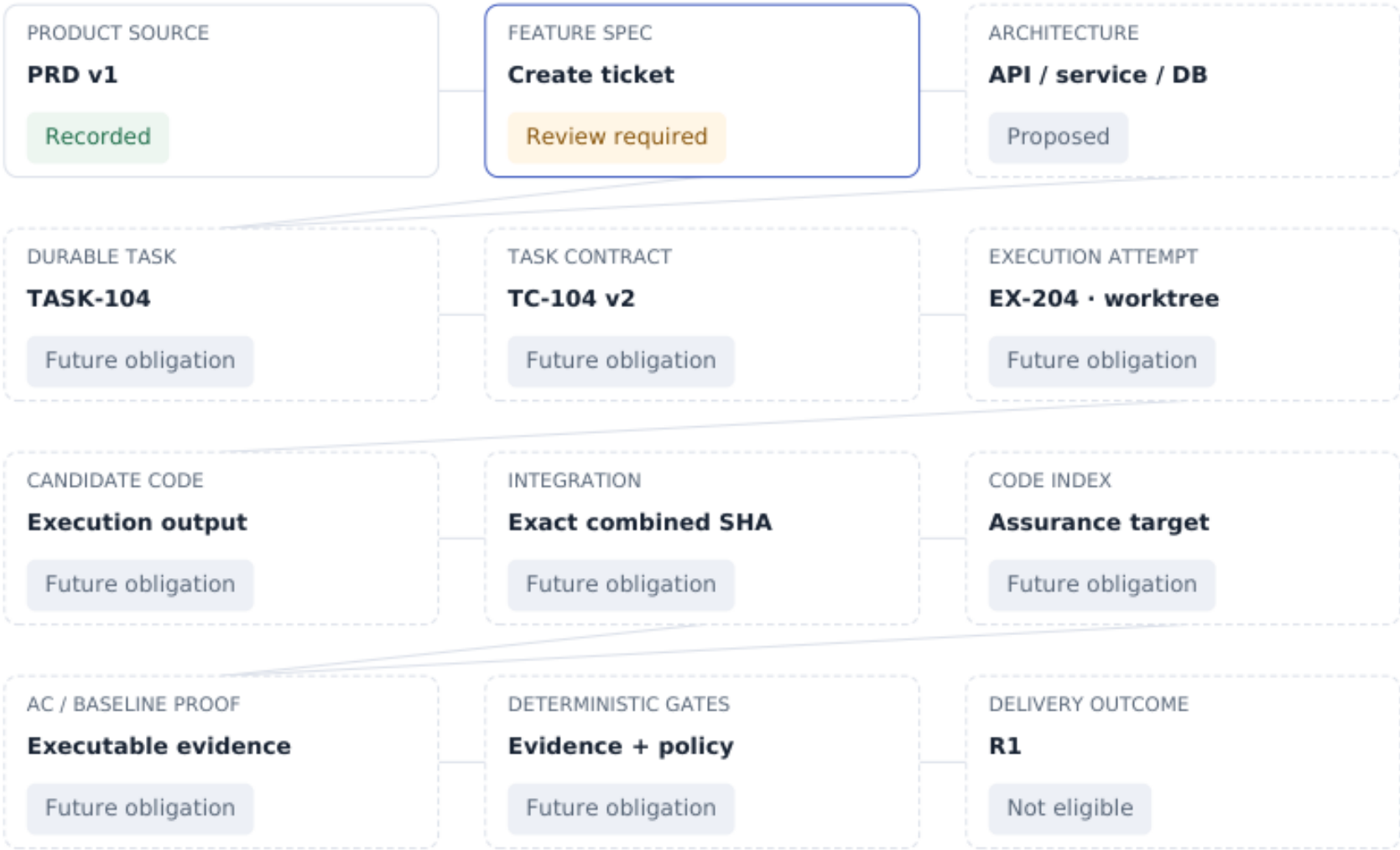
Scope and architecture are reviewable

State → reason → evidence → allowed action → lineage

Why this state?

Control-plane map

Lifecycle is above; object relationships are the workspace



Select object → inspect reason / provenance → open contextual detail

Object inspector

Version-bound context and reason

SPEC-001 v1

Review proposed Create Ticket behaviour

Authority

Olympus control-plane record

Version / origin

ProductSource v1 · proposed spec

Commit scope

9e31ab7

Workspace

Source-backed product model

Why this state?

Review the proposed behavioural contract before planning writable implementation.

Open contextual detail

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-002 · Brownfield Onboarding

New delivery cycle

SUPPORTDESK / DELIVERY CYCLE

Delivery Cycle

Establish a trusted model of the existing repository

✓ RECON ✓ CODE INDEX ● RECOVERED SPEC ○ BASELINE ○ READINESS ○ READY

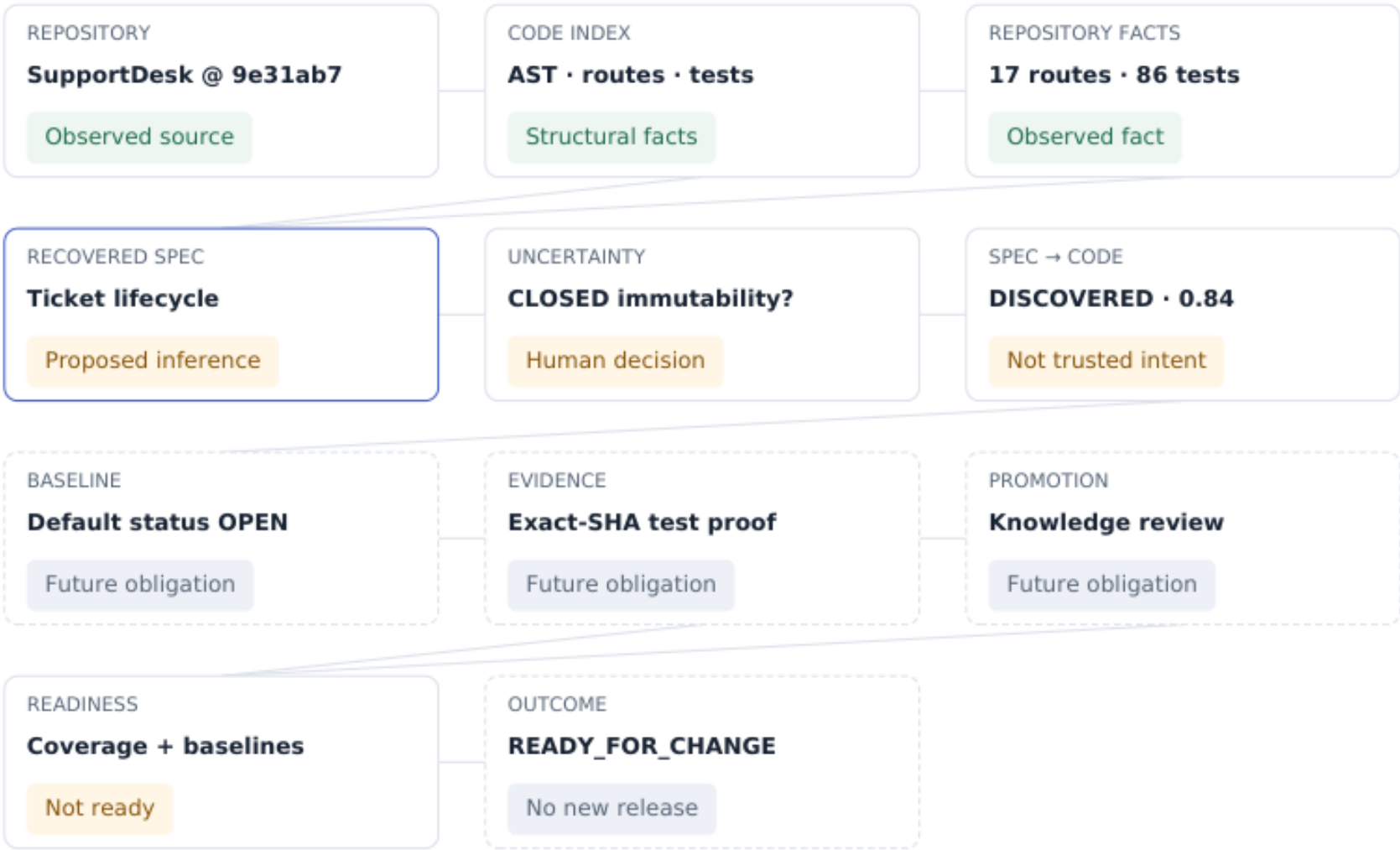
Recovered intent needs evidence-backed review

State → reason → evidence → allowed action → lineage

Why this state?

Control-plane map

Lifecycle is above; object relationships are the workspace



Select object → inspect reason / provenance → open contextual detail

Object inspector

Version-bound context and reason

REC-017

Recovered ticket lifecycle

Authority
Olympus control-plane record

Version / origin
DISCOVERED · confidence 0.84

Commit scope
9e31ab7

Workspace
Read-only repository

Why this state?

Observed behaviour does not establish intended product truth. Promotion preserves evidence and reviewer.

Open contextual detail

WORKSPACE

01 Project Overview

02 Delivery Cycle

03 Product / Specs

04 Task DAG

05 Execution Inspector

06 Code Intelligence

07 Traceability

08 Impact Explorer

09 Assurance

10 Release

Integrations

Audit history

SS Engineering lead

Project operator

DELIVERY CYCLE

DC-004 · Bug Fix

New delivery cycle

SUPPORTDESK / DELIVERY CYCLE

Delivery Cycle

Reject updates to CLOSED tickets safely

✓ TRIAGE

● REPRODUCTION

○ EXPECTED BEHAVIOR

○ ROOT CAUSE

○ DEVELOPMENT

○ INTEGRATION

○ REGRESSION

○ ASSURANCE

○ RELEASE

○ COMPLETE

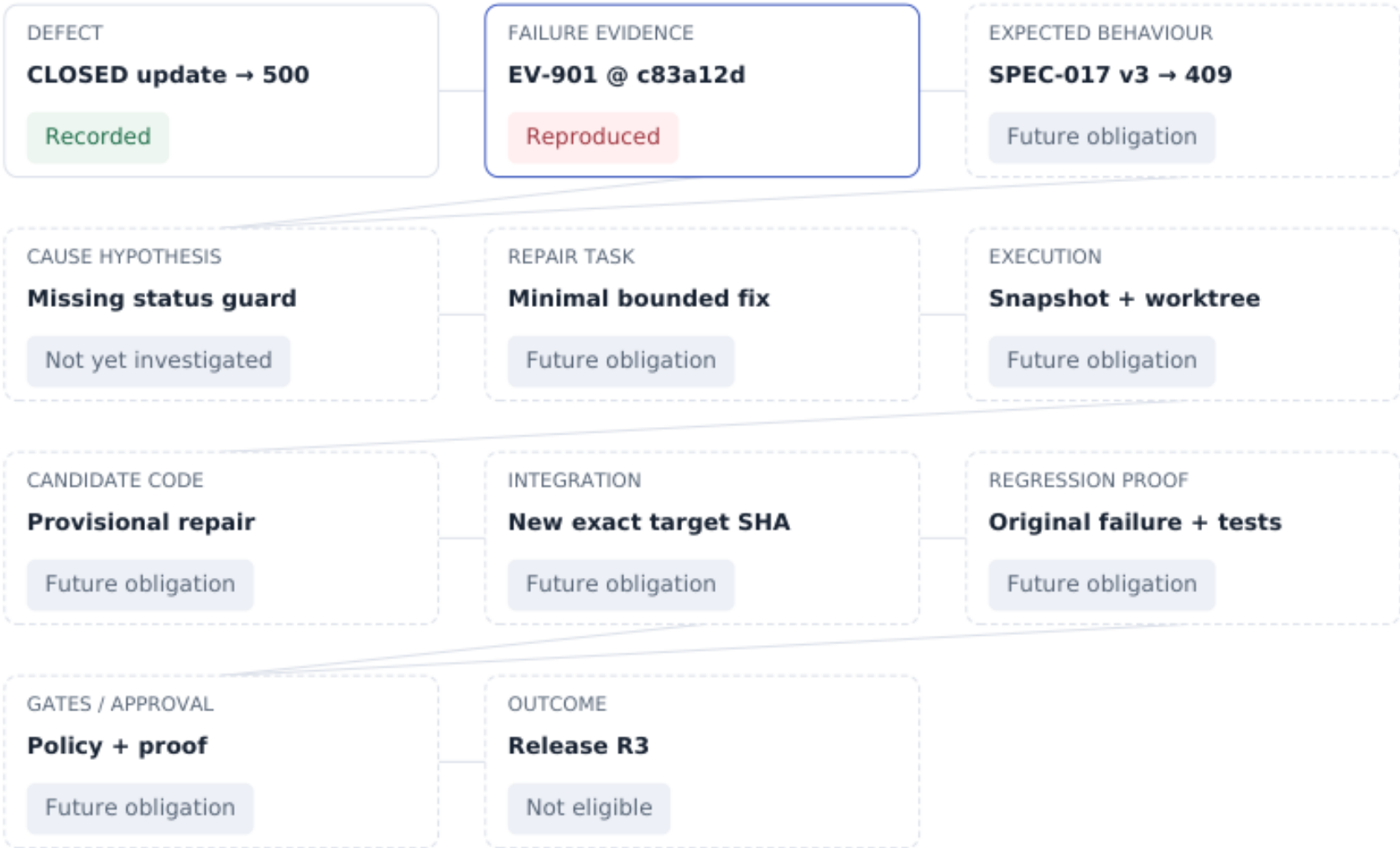
Failure evidence exists before the repair

State → reason → evidence → allowed action → lineage

Why this state?

Control-plane map

Lifecycle is above; object relationships are the workspace



Select object → inspect reason / provenance → open contextual detail

Object inspector

Version-bound context and reason

EV-901

HTTP 500 recorded before repair

Authority

Olympus control-plane record

Version / origin

EV-901 · REPRODUCTION

Commit scope

c83a12d

Workspace

Reproduction diagnostic context

Why this state?

Failure history is retained. Expected behaviour and independent after-proof are separate obligations.

Open contextual detail

Design direction and scope

Olympus is a governed software-delivery command center. The Delivery Cycle control-plane graph is the primary operational workspace. Overview, Tasks, Executions, Code and Assurance are contextual drill-downs, with direct navigation retained for expert users.

The primary operator is a Product Owner / Engineering Lead. Every view should answer: where are we, what is happening, what requires attention, why, and what happens next. The interaction order is state → reason → evidence → permitted action → traceability.

This pack contains proposed visual designs, ten screen contracts, navigation rules, all four canonical lifecycle walkthroughs, component specifications, exceptional states and engineering handoff. SupportDesk names, IDs, counts, SHAs and code excerpts are illustrative fixtures.

Source requirements: Olympus MVP Architecture & Implementation Plan v1.2 and Technical Implementation Specification v1.0. Layout, colors, routes and component names are design proposals. This is not a Figma file, production frontend or live agent demonstration. Local preview actions do not establish real evidence, approvals or releases.

Information architecture and navigation

Open Project Overview to establish the released baseline and active objective. Open the active Delivery Cycle as the operational home. Its lifecycle ribbon shows journey state; its graph shows connected authoritative records; its inspector shows object context, reasons, provenance and permitted commands.

Click a graph node to select it without mutation. Open contextual detail to enter its canonical screen. Preserve project, cycle, selected object, version, index scope and target SHA in navigation. A persistent Back to cycle action restores selection and graph position.

The ten canonical screens are not a mandatory linear wizard. Product / Specs, Task DAG, Execution, Code, Traceability, Impact and Assurance branch from the Delivery Cycle map. Release is the governed outcome view. Brownfield uses the same outcome view for readiness handoff.

Global utilities: New delivery cycle opens intent-specific intake; Ask Olympus opens a state-backed explanation / typed-command panel; Integrations exposes authenticated inbound events, governed outbound actions and reconciliation; Audit exposes attributable immutable history. These are auxiliary surfaces, not new delivery journeys.

Agent names appear only within capability/execution context. They do not organize global navigation. Product, work, execution, intelligence and assurance objects organize the map.

Canonical screen-to-journey variations

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Overview	No released baseline initially; R1 target	Existing repo / release; trusted baseline target	R1 baseline and R2 target	R2 baseline and R3 target

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Delivery Cycle	Intent → new product / code → proof	Code → recovered knowledge → readiness	Delta → impact → code → regression proof	Failure → intended behaviour → repair → proof

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Product / Specs	Decompose PRD; scope / architecture approval	Observed vs recovered vs canonical intent	Versioned behavioural / technical delta	Resolve expected behaviour or checkpoint

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Task DAG	Implementation and verification dependencies	Discovery / reasoning / baseline work; no feature build implied	Minimal impact-aware tasks	Minimal bounded repair task(s)

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Execution	Planning and writable attempts	Mostly discovery/reasoning; isolated baseline work if permitted	Bounded code-change attempts	Reproduction, diagnostic and repair attempts

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Code Intelligence	New integrated code and generated lineage	Existing canonical structure; discovered intent links	Affected symbols and incremental re-index	Failure path and repaired symbol

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Traceability	Source to verified R1	Code to recovered intent / baseline / readiness	Request and delta to safe R2	Defect + before/after proof to R3

Canonical screen-to-journey variations — continued

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Impact	Architecture / implementation boundaries	Readiness gaps and unresolved uncertainty	Affected symbols / contracts / tests / baselines	Probable cause + minimal repair scope

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Assurance	New mandatory AC proof and review	Executable baselines / coverage / readiness	New AC proof + impacted old behaviour	Original failure now passes + regression + baseline proof

Screen	Greenfield	Brownfield	Feature Change	Bug Fix
Release	Verified R1 manifest; optional deployment	Readiness handoff; no new release	Verified R2 manifest	Verified R3 manifest

Greenfield Build — end-to-end UI journey

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
DISCOVERY	Create cycle / Product	Upload PRD or product description	Immutable ProductSource, version and hash	Source enters graph; extraction is a proposal

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
PRODUCT_MODEL	Product / Specs	Resolve questions; review scope	Capabilities, Features, FeatureSpecs, REQ / stories / ACs	Source-to-intent links, proposed vs approved versions

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
ARCHITECTURE	Impact / Product	Review architecture where policy requires	Architecture baseline, contracts, ImplementationSpecs	Project-wide constraints and feature realization boundary

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
PLANNING	Task DAG	Inspect bounded plan and dependencies	Validated DAG and compiled TaskContracts	Why task is ready / blocked; required outputs

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
DEVELOPMENT	Execution Inspector	Observe; answer checkpoint questions	Snapshots, leases, isolated worktrees, artifacts, candidates	Live attempt events and provisional code; history preserved

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
INTEGRATION	Code Intelligence	Inspect combined candidate / conflicts	Exact integrated SHA and canonical index / links	Candidates converge; assurance target differs from released baseline

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
ASSURANCE	Assurance	Inspect gaps; request verification or repairs	AC proof, independent review, deterministic gates	Same-SHA evidence and missing obligations

Greenfield Build — end-to-end UI journey — continued

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
RELEASE	Release	Review required approval; execute when eligible	Versioned R1 manifest and outcome bundle	All predicates including approvals evaluated by server

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
COMPLETE	Overview / Traceability	Inspect released outcome and continue	R1 and complete lineage; deployment optional	Persistent project remains available for later changes

Brownfield Onboarding — end-to-end UI journey

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
RECON	Create cycle / Delivery	Register repo, branch, exact SHA	Immutable source reference and deterministic facts	Code inputs precede product recovery

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
CODE_INDEX	Code Intelligence	Inspect structural code and tests	AST / routes / schemas / tests at pinned SHA	Canonical structural facts; product intent still unknown

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
RECOVERED_SPEC	Product / Traceability	Review evidence, uncertainty and provenance	ObservedBehaviour, RecoveredSpec proposals, discovered links	FACT vs INFERENCE vs UNCERTAINTY visible

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
BASELINE	Assurance / Task DAG	Review baseline proposal; request bounded runs	Behavioural baselines and exact-repository-SHA proof	Observed accidental behaviour is not silently blessed

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
READINESS	Assurance / Impact	Resolve critical gaps; promote trusted intent	Human/evidence-backed promotions, readiness assessment	Coverage + baseline evidence + unresolved uncertainty

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
READY	Release handoff / Overview	Start Feature Change or Bug Fix	Trusted baseline B1 + READY_FOR_CHANGE	Existing release unchanged; no new release / deployment

Feature Change — end-to-end UI journey

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
INTAKE	Create cycle / Delivery	Submit requested behaviour	ChangeRequest linked to existing project context	Request enters existing product/code graph

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
SPEC_DELTA	Product / Specs	Review v1 → v2 and required approval	Versioned FeatureSpec / ImplementationSpec delta	Stable feature identity; approved behaviour change

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
IMPACT_ANALYSIS	Impact Explorer	Inspect paths, tests and baselines	Affected code / contracts / obligations, rationale	Direct structural impact distinguished from inferred expansion

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
PLANNING	Task DAG	Inspect minimal safe plan	Validated tasks and immutable contracts	Dependencies and scope precede admission

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
DEVELOPMENT	Execution Inspector	Observe bounded work / decisions	Attempt history, snapshots, worktree, candidate commits	Producer output is provisional

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
INTEGRATION	Code Intelligence	Inspect combined change / re-index	Exact integrated candidate and refreshed links	Canonical assurance index advances; released baseline remains R1

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
ASSURANCE	Assurance	Verify new ACs + impacted old behaviours	New proof, regression / baseline proof and gates	Both new and existing behaviour proven at candidate SHA

Feature Change — end-to-end UI journey — continued

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
RELEASE	Release	Approve where required; execute when eligible	R2 manifest and outcome	Release is independent of optional deployment

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
COMPLETE	Overview / Traceability	Inspect lineage and continue	R2 becomes persistent project baseline	Historical R1, versions and failures retained

Bug Fix — end-to-end UI journey

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
TRIAGE	Create cycle / Delivery	Report defect with reproduction steps	Defect with source and severity context	Failure enters the graph as a reported observation
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
REPRODUCTION	Assurance / Execution	Inspect failing scenario at before SHA	REPRODUCTION evidence before repair where feasible	Reproduced defect is a successful investigation, not acceptance PASS
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
EXPECTED_BEHAVIOR	Product / Specs	Resolve intent from canonical spec / baseline	Expected behaviour with authoritative provenance	Unknown expectation checkpoints for human/spec decision
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
ROOT_CAUSE	Impact Explorer	Review probable faulty path and minimal scope	Reasoned cause hypothesis with trace + code evidence	Probability is separate from proven cause
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
DEVELOPMENT	Task / Execution	Observe minimal bounded repair	Repair contract, attempt, worktree and candidate	No unrelated feature or architecture redesign
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
INTEGRATION	Code Intelligence	Inspect combined repaired SHA / re-index	New integration candidate and refreshed links	Before and after SHAs are explicit
Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
REGRESSION	Assurance	Re-run original failure + regression + baselines	After evidence bound to repaired candidate	Failing history retained; new independent proof required

Bug Fix — end-to-end UI journey — continued

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
ASSURANCE	Assurance	Inspect required review, findings and gates	Warden/Sentinel results finalized by Olympus	No producer self-certification

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
RELEASE	Release	Review required approval; execute when eligible	R3 manifest and outcome	Verified repaired SHA pinned

Lifecycle stage	Surface	Operator action	Persisted output	Control-plane visualization
COMPLETE	Traceability / Overview	Inspect forensic and delivery history	Defect → failure → intent → repair → proof → R3	Failure evidence remains reachable after repair

Graph grammar and progressive disclosure

Group objects into Intent / Product, Planned Work, Execution / Candidate, Canonical Code, Evidence / Decisions, and Outcome. Reveal the neighborhood relevant to the current cycle rather than a global repository hairball. Collapsed groups display record counts and highest-priority blocker; expansion reveals typed objects.

Use explicit typed relationship labels in the selected-edge inspector: DERIVED_FROM, IMPLEMENTS, DEPENDS_ON, EXECUTED_AS, PRODUCED, INTEGRATED_IN, VERIFIED_BY, APPROVED_BY and CONTAINED_IN. These are proposed presentation labels; map them to real persisted relation types during implementation.

Solid node: materialized authoritative record. Dashed node: future obligation, never represented as a completed artifact. Selection is an outline; state is a textual chip. Solid structural edges and labelled inferred relationships differentiate deterministic observations from semantic interpretation.

Click a status to open the same Why inspector. It shows required conditions, passed / unmet predicates, blocking record IDs, policy key/version, input versions, evaluation time and the allowed next command. A model explanation may summarize this result but cannot replace it.

Real graph interactions: focus on selected neighborhood, expand a group, pan/zoom on desktop, accessible list mode and breadcrumb reset. The in-conversation concept implements node selection and local navigation; production pan/zoom, list projection and large-graph virtualization remain engineering work.

Lineage is bidirectional. Principal symbols map to specifications; helpers inherit context through structural relationships rather than receiving duplicate feature links. Preserve uncertainty, confidence and supporting code/test evidence on discovered edges.

Repository, execution and evidence truth

Concept	Required visible distinction	Consequence
Task vs Execution	Durable work vs one immutable attempt	Failures / retries remain reachable; task completion requires validated outputs

Concept	Required visible distinction	Consequence
Snapshot vs current project	Pinned inputs / contract / base SHA vs later state	Input changes can make attempt stale or require revalidation

Concept	Required visible distinction	Consequence
Candidate index	Execution-scoped provisional code	Never imply released / canonical project baseline

Concept	Required visible distinction	Consequence
Canonical assurance index	Exact current IntegrationCandidate SHA	Independent gates and final proof target this SHA

Concept	Required visible distinction	Consequence
Released baseline	Exact manifest SHA of latest recorded release	Does not advance solely because a candidate was integrated

Concept	Required visible distinction	Consequence
Structural fact vs intent	AST / route / schema observation vs intended behaviour	Recovered product truth requires evidence-backed review / promotion

Concept	Required visible distinction	Consequence
Evidence freshness	Target SHA / source version / obligation explicit	Old-SHA proof is historical; cannot satisfy current target

Concept	Required visible distinction	Consequence
Technical checks vs eligibility	Proof ready, approval pending, eligible, released are distinct	Required approvals are part of final eligibility

Repository, execution and evidence truth — continued

Concept	Required visible distinction	Consequence
Release vs Deployment	Recorded delivery outcome vs optional external deploy	No green deployment badge without independent deployment result

Shared surfaces and command interactions

Surface	Required fields	Interaction
New cycle intake	Journey, project, versioned source; repo branch/SHA for onboarding; requested behaviour for change; reproduction steps/observed result for defect	Validate inputs; submit typed create/intake command; acknowledge persisted cycle and navigate to map

Surface	Required fields	Interaction
Why inspector	Status, actor/service, evaluated version, policy, predicate checks, blocking IDs, linked proof, next action	Read-only by default; permitted actions still go through command service

Surface	Required fields	Interaction
Approval review	Decision type/scope/version/SHA, reason, impact, risk, evidence, actor, rationale	Approve / reject / request changes; server validates expected version and required authorization

Surface	Required fields	Interaction
Clarification checkpoint	Question ID, unknown intent, authoritative sources already checked, required answer, waiting execution	Answer creates durable decision/spec version where needed; resume/replan under policy

Surface	Required fields	Interaction
Ask Olympus	Current cycle and object scope, linked reasons/proof, structured suggested command	Explain existing records; mutation becomes explicit typed command with normal authorization

Surface	Required fields	Interaction
Integrations	Inbound source/version/signature/correlation/idempotency; outbound action/scope/policy/result	Unknown mutation outcome → reconcile before retry; show checkpoint/recovery state

Surface	Required fields	Interaction
Audit history	Actor, command, time, version, correlation ID, before/after references, decision rationale	Filter/read history and traverse immutable failures / approvals / connector results

Exceptional states that must be designed

State / trigger	Visible reason and consequence	Permitted next action
Unknown expected behaviour	Bug repair waits; no invented intended semantics	Resolve canonical spec / baseline or request human decision
State / trigger	Visible reason and consequence	Permitted next action
Proposed Brownfield inference	Confidence and code evidence shown; not canonical intent	Review, request evidence, reject or promote with authority
State / trigger	Visible reason and consequence	Permitted next action
Dependency blocked	List unmet task/artifact/approval predicate	Open blocker; schedule only after eligibility recomputation
State / trigger	Visible reason and consequence	Permitted next action
Out-of-scope tool action	Denied action logged with contract scope / policy	Request bounded scope change / approval, never silently retry wider
State / trigger	Visible reason and consequence	Permitted next action
Execution failed / timed out	Attempt and artifacts retained; task not auto-completed	Inspect failure; create policy-compliant retry
State / trigger	Visible reason and consequence	Permitted next action
Checkpoint / lease expiry	Waiting dependency/question and worker ownership visible	Resolve and resume/replan; preserve attempt history
State / trigger	Visible reason and consequence	Permitted next action
Stale snapshot / changed base	Pinned inputs differ from authoritative versions	Revalidate or create new contract/snapshot/attempt
State / trigger	Visible reason and consequence	Permitted next action
Integration conflict	Conflict is a Finding, not hidden autonomous merge success	Create remediation work and reintegrate
State / trigger	Visible reason and consequence	Permitted next action
Missing or old-SHA evidence	Obligation and wrong/current SHA visible	Request exact-target verification

Exceptional states that must be designed — continued

State / trigger	Visible reason and consequence	Permitted next action
Assurance failure	Gate cannot pass; finding linked to proof	Repair loop then targeted revalidation

State / trigger	Visible reason and consequence	Permitted next action
Approval pending / rejected	Technical readiness does not imply eligibility	Review decision; revise scope when rejected

State / trigger	Visible reason and consequence	Permitted next action
Connector outcome unknown	Possible external mutation; do not blind retry	Reconcile provider state with external correlation

State / trigger	Visible reason and consequence	Permitted next action
Event stream disconnected	Last authoritative refresh timestamp + disconnected state	Refetch current state; avoid optimistic green completion

State / trigger	Visible reason and consequence	Permitted next action
Empty / loading / forbidden	Explain unavailable object or action, preserve context	Upload/register source, retry fetch or use authorized account

Figma component and visual-system handoff

Frame presets: desktop 1440 × 1000, compact desktop 1024 × auto, tablet 768 × auto, mobile 390 × auto. Use auto-layout throughout; 8 px spacing base, 16–24 px content padding, 8 px surface corners, 6 px control corners. Design intent is restrained, information-dense engineering tooling.

Wide desktop: navigation 180–200 px; main workspace flexible; inspector 280–320 px. Compact desktop: navigation 145–166 px; inspector moves below graph before labels become illegible. Mobile: navigation reflows, graph becomes vertical or accessible list, tables retain contained horizontal scrolling when columns cannot fit.

Typography: Inter/system sans; page title 24 px / medium; section 16 px / medium; body 13–14 px; annotations minimum 11 px. SHAs, entity IDs, timestamps and code use a readable monospace face. Do not shrink entire graphs to fit.

Reusable components: AppShell, ProjectContextBar, CyclePicker, LifecycleRibbon, AttentionQueue, ControlPlaneCanvas, GraphGroup, ObjectNode, RelationInspector, WhyInspector, StatusBadge, ProvenanceBadge, ShaScopeBanner, TaskContractCard, AttemptHistory, ExecutionTimeline, EvidenceMatrix, EligibilityChecklist, VersionDiff, ApprovalDialog, CheckpointDialog, IntakeForm, OutcomeManifest and IntegrationReconciliation.

Each component needs loading, empty, unavailable, permission-denied and stale variants where meaningful. ObjectNode has materialized/future/selected variants. StatusBadge covers running, blocked, waiting, failed, stale, passed and complete with both text and color. ProvenanceBadge covers FACT, INFERENCE, UNCERTAINTY, ASSUMPTION and DECISION, plus exact supporting source references.

Approval dialog auto-layout: scope + exact version/SHA; reason and impact; supporting records; risk; required rationale; Reject / Request changes / Approve. Full keyboard operation, focus containment and focus restoration are required. Do not put approval behind a chat transcript.

Maintain Figma pages: Foundations; Shared Components; App Shell; Canonical Screens; Four Journey Flows; Exception States; Interaction Prototype; Engineering Notes. Name frames S01–S10 and journey snapshots GF/BF/FC/BG + lifecycle name. Connect graph-node components to contextual detail frames; preserve Back to cycle flow.

Token	Light	Dark	Usage
Background	#F5F6F8	#11151C	Workspace canvas

Token	Light	Dark	Usage
Surface	#FFFFFF	#191E27	Opaque panels / dialogs

Token	Light	Dark	Usage
Text	#182333	#E3E9F2	Primary content

Figma component and visual-system handoff — continued

Token	Light	Dark	Usage
Secondary text	#596779	#A4B1C3	Supporting metadata

Token	Light	Dark	Usage
Border	#DDE3EB	#343E4E	Quiet structure

Token	Light	Dark	Usage
Active	#3D59BF	#A6B7FF	Selection / running labels

Token	Light	Dark	Usage
Success	#21704E	#9AD7B7	Passed conditions

Token	Light	Dark	Usage
Attention	#8A560C	#E9C38B	Waiting / review

Token	Light	Dark	Usage
Failure	#A23842	#EF9AA3	Failure evidence / rejected action

Engineering handoff and acceptance

Use documented Next.js / TypeScript / React / shadcn/ui, TanStack Query and SSE. The frontend projects authoritative control API responses. SSE invalidates/refetches server state; authoritative completion is not fabricated in a client reducer. Client route proposals in this pack are not assertions that backend endpoints already exist.

Read models need entity type/ID/version, status, source/provenance, policy reason, input/candidate/index SHA, evidence links and permitted commands. Graph edges need actual relation type, source/target identity, provenance, confidence where relevant, source evidence and version scope. Server-specific schemas should be finalized before frontend wiring.

Commands use documented control API patterns: POST delivery-cycles/{id}/commands/{command}, POST approvals/{id}/decision, execution cancellation, spec approval and release approve/execute. Submit actor context, expected version/state and idempotency metadata as supported. Never PATCH lifecycle status directly from a view.

Gate results, release eligibility and readiness are server computations. Separate independent Warden/Sentinel findings from final platform Gate state. Pin final assurance to the IntegrationCandidate commit. Show denial reasons and current authoritative state after rejected commands.

Graph and table views must share the same server records. Add deep links from Feature/Spec → Task/Execution → CodeEntity → Evidence/Gate → Release and backward. A reviewer should reach raw supporting proof from any PASS indicator. Avoid exposing secrets in model metadata, tool logs or source snapshots.

Validation acceptance: all four lifecycles preserve state clarity, control-plane reasons, execution transparency, traceability and repository truth. Rejected approval, stale execution, missing evidence, integration conflict and connector unknown result have reviewable UI states. Brownfield finishes at readiness without new release. Bug Fix retains before-failure evidence and exact-SHA after proof.

The concept preview supports ten screens, four journeys, stage snapshot selection, node inspection, local explanation/approval/intake/integration/audit overlays and typed-command previews. It is fixture-driven, not backend-integrated. Screen navigation chooses a representative stage snapshot in the preview; production navigation preserves authoritative state. A local JavaScript harness checked rendering and core interactions across all 40 journey/screen combinations plus auxiliary dialogs. Browser-level layout validation was unavailable in this environment; responsive and production accessibility checks remain required.

Source alignment: architecture invariants are retained. Figma authoring, real graph pan/zoom/virtualization, backend data wiring, agent execution, live-LLM journey proof, real approvals, connector actions and deployments are subsequent implementation work. No UI concept should be mistaken for the technical definition of done.

Source cross-reference

Requirement	Architecture plan v1.2	Technical specification v1.0
Control-plane authority / operator	§§1-4, 23	§§1, 5, 7, 25

Requirement	Architecture plan v1.2	Technical specification v1.0
Versioned product semantics	§§4.1, 14	§§5.2, 6, 14

Requirement	Architecture plan v1.2	Technical specification v1.0
Task / contract / execution	§§5, 7-9	§§8-12

Requirement	Architecture plan v1.2	Technical specification v1.0
Canonical index / traceability	§§13-14	§§13-14, 17

Requirement	Architecture plan v1.2	Technical specification v1.0
Greenfield / Brownfield / change / bug	§§15-18, 26.1	§§19-22

Requirement	Architecture plan v1.2	Technical specification v1.0
Evidence / gates / outcome	§§11-12	§§18, 23

Requirement	Architecture plan v1.2	Technical specification v1.0
Inbound / outbound / reconciliation	§§20.1-20.4	§§15-16

Requirement	Architecture plan v1.2	Technical specification v1.0
Frontend / visibility / live proof	§§23-24, 26	§§24-27, 31-32

S01 · Project Overview

Operator goal

Understand persistent project truth, the active objective and the next operator decision.

Screen composition

Released baseline and active cycle summary; attention queue; compact coverage with explicit denominators; cycle history.

Actions and navigation

Open active delivery cycle; create cycle; inspect blocker.

Authoritative objects

Project, DeliveryCycle, Release, index version, approval / blocker summaries.

State and truth requirement

Do not use an invented overall completion percentage. If coverage appears, label whether it is specification mapping, mandatory AC proof or executable coverage.

Proposed route

/projects/:projectId

Source alignment

Architecture §23; Technical §25

S02 · Delivery Cycle

Operator goal

Operate the current change through a visible control plane. This is the central workspace.

Screen composition

Cycle header; journey-specific lifecycle ribbon; attention reason; focused relationship graph; selected-object inspector.

Actions and navigation

Select object; explain state; open contextual drill-down; request allowed typed command.

Authoritative objects

DeliveryCycle, product / work / execution / intelligence / assurance records.

State and truth requirement

Top ribbon represents lifecycle, graph represents connected objects and inspector represents authorization / reasons. Viewing a node does not issue a command.

Proposed route

/projects/:projectId/cycles/:cycleId

Source alignment

Architecture §§6, 7, 13, 14; Technical §§7, 25

S03 · Product / Specs

Operator goal

Review intended behaviour, technical realization, versions and provenance.

Screen composition

Capability / feature tree; behavioural specification; REQ / story / AC children; ImplementationSpec; version diff; architecture link.

Actions and navigation

Resolve clarification; approve scope / spec when policy requires; review recovered intent; request changes.

Authoritative objects

ProductSource, Capability, Feature, FeatureSpec, REQ, UserStory, AC, ImplementationSpec, Architecture.

State and truth requirement

Behavioural truth and implementation design remain distinct. Architecture is project-wide; implementation specs cannot silently redefine it. Brownfield recovery is proposed until reviewed.

Proposed route

/projects/:projectId/features/:featureId/specs/:version

Source alignment

Architecture §§4.1, 13.2, 14.6; Technical §§5.2, 6

S04 · Task DAG

Operator goal

Explain dependency order, admission and durable work boundaries.

Screen composition

Dependency graph and accessible list; work type, task state, dependencies; immutable TaskContract; attempt history.

Actions and navigation

Inspect contract, dependencies and outputs; follow active or historical execution.

Authoritative objects

Task, TaskDependency, TaskContract, specification versions, Execution references.

State and truth requirement

A task is durable; an execution is one attempt. A completed unit test or producer declaration does not imply task, product or release completion.

Proposed route

/cycles/:cycleId/tasks?selected=:taskId

Source alignment

Architecture §§5, 7.1; Technical §8

S05 · Execution Inspector

Operator goal

Make bounded autonomous work and recovery attributable.

Screen composition

Attempt header; contract, snapshot, base SHA, lease and worktree; governed tool events; candidate diff; model usage metadata.

Actions and navigation

Inspect events / diff; request cancellation; answer checkpoint question; resume using authorized command.

Authoritative objects

Execution, ExecutionSnapshot, TaskContract, Lease, Worktree, ToolAction, Artifact, CandidateCommit.

State and truth requirement

Retry creates a new attempt where required. Preserve failed attempts and old snapshots. Show configured model alias and observed provider metadata, not a hard-coded model name.

Proposed route

/executions/:executionId

Source alignment

Architecture §§7–9; Technical §§9, 10, 12, 24

S06 · Code Intelligence

Operator goal

Inspect structural repository truth and its relationship to specification intent.

Screen composition

Pinned index scope and SHA; file/symbol tree; principal symbol; calls / imports / schemas / tests; provenance and confidence.

Actions and navigation

Traverse to spec or test; compare provisional execution index with canonical integrated/released index.

Authoritative objects

CodeIndexVersion, CodeEntity, CodeRelation, SpecCodeLink.

State and truth requirement

Label provisional candidate code, canonical assurance target, and released baseline separately. Canonical structural facts do not automatically make discovered product intent canonical.

Proposed route

/projects/:projectId/code?index=:indexId&symbol;=:symbolId

Source alignment

Architecture §§13.1, 14.3–14.8; Technical §§13, 14

S07 · Traceability

Operator goal

Answer why code exists, who or what produced it, what proves it and which outcome contains it.

Screen composition

Narrative lineage with bounded expansion; exact versions; forward/back traversal; selected-object evidence and source provenance.

Actions and navigation

Follow record or edge in either direction; inspect origin, confidence and supporting evidence.

Authoritative objects

Source → specs → task / contract → execution / snapshot → candidate / integration → code → tests / evidence → gates / approval → release.

State and truth requirement

Keep historical, superseded and inferred links visible with labels. Discoveries are distinct from GENERATED_LINEAGE and HUMAN_CONFIRMED links.

Proposed route

/projects/:projectId/lineage?selected=:entityId&cycle;=:cycleId

Source alignment

Architecture §§13, 14; Technical §14

S08 · Impact Explorer

Operator goal

Establish a bounded change or repair scope before writable execution.

Screen composition

Existing spec and delta; affected symbols / schemas / API contracts; graph path rationale; selected tests / baselines; risk and architecture impact.

Actions and navigation

Inspect direct vs inferred impact; review bounded plan; escalate architecture delta; open resulting task plan.

Authoritative objects

ChangeRequest, Defect, ImpactAssessment, CodeRelation, SpecCodeLink, Baseline, ImplementationSpec.

State and truth requirement

Greenfield uses architecture / realization scope. Brownfield uses uncertainty / readiness gaps. Bug Fix uses failure, expected behaviour and probable cause; inference is not proven fact.

Proposed route

/cycles/:cycleId/impact

Source alignment

Architecture §§17–18; Technical §§21–22

S09 ▪ Assurance

Operator goal

Inspect independent evidence, gaps, findings and deterministic gate state.

Screen composition

Exact target SHA; AC / baseline evidence matrix; Warden review and Sentinel obligations; findings and remediation loop.

Actions and navigation

Inspect raw proof; request verification; open finding / repair work; explain gate.

Authoritative objects

IntegrationCandidate, Evidence, AcceptanceCoverage, Finding, Gate, Baseline.

State and truth requirement

Agents provide recommendations and evidence; Olympus finalizes gates. Old-SHA proof cannot satisfy current candidate. Brownfield uses baseline / readiness conditions, not release gates.

Proposed route

/cycles/:cycleId/assurance?candidate=:candidateId

Source alignment

Architecture §§11–12; Technical §18

S10 · Release

Operator goal

Review exact-SHA eligibility, required decisions and the recorded delivery outcome.

Screen composition

Manifest; verified candidate SHA; checklist with exact blockers; approvals; outcome history; separate optional deployment.

Actions and navigation

Review release approval; request execute only when all server eligibility conditions pass; open lineage / proof.

Authoritative objects

ReleaseManifest, IntegrationCandidate, Gate, Evidence, Approval, Release; optional deployment record.

State and truth requirement

Technical checks complete, approval pending, eligible and released are distinct states. Approval cannot replace proof. Brownfield displays readiness handoff and existing release, with no new release command.

Proposed route

/cycles/:cycleId/outcome

Source alignment

Architecture §12; Technical §§18–22, 23